Backend checklist, explained

SMARTCART

What is in place, how it works, and where it helps the project

A practical tour of the backend checklist: what each item does, where SmartCart implements it, why it is useful, and a compact code example. The examples are excerpts; use the linked paths (relative to Backend/) to see the full implementation.

Coverage at a glance

IMPLEMENTED

Of the 26 requested checklist items: 20 are implemented, 5 are partial, and 1 is pending. Partial means the capability exists but has a stated scope or compatibility limitation. The internet deployment item remains pending because this workspace contains local Docker configuration, not a live hosted deployment.

How to read the labels

IMPLEMENTED

IMPLEMENTED means the behavior exists in code and, where appropriate, is covered by the current gateway tests. PARTIAL means it exists with a meaningful limitation described here. PENDING means the checklist outcome is not present yet.

Project map

IMPLEMENTED

The API gateway is the public entry point for REST, GraphQL, validation, throttling, idempotency, and upstream resilience. Product, order, payment, and other services own their domain logic. MongoDB stores application documents; MySQL stores gateway idempotency and audit records. Docker Compose connects the local services.

Status reflects the source in this workspace; it does not imply production hosting, live third-party credentials, or a production-scale distributed setup.

HTTP API behavior

HTTP methods, /v1, resource URLs

PARTIAL

The gateway exposes version-prefixed aliases such as /v1/api/products while retaining the existing /api routes for compatibility. The public URLs are mostly resource-shaped. One adapter maps a few public resource-style actions (for example, PATCH .../cancellation) to older internal service paths and methods, so method semantics are not uniformly owned by every service. See api-gateway/src/app.js.

```
app.use((req, res, next) => {
  req.apiPath = req.path.replace(/^\/v1(?=\/api\/)/, "");
  next();
});
```

Idempotency keys on POST

IMPLEMENTED

For selected create operations (products, orders, cart items, payment creation, returns, and refunds), the gateway requires an Idempotency-Key. It remembers the first response, replays matching retries, rejects a reused key with a changed payload (409), and reports an in-progress duplicate (409). This helps clients retry after a dropped response without accidentally creating duplicates. It is not applied to every POST, such as login or webhooks.

```
await pool.execute(
  "INSERT INTO api_idempotency (...) VALUES (?,?,?,?)",
  [scope, key, route, method]
);
```

AJV JSON Schema validation

IMPLEMENTED

The gateway compiles JSON Schemas with AJV and checks selected request bodies before forwarding them. Invalid product, order, cart, payment, registration, or login input gets rejected early with a 400 response. This protects service code from malformed requests; domain rules still belong in the owning service. See api-gateway/src/schemas.js and app.js.

```
const ajv = new Ajv({ allErrors: true });
const validate = ajv.compile(schemas.productCreate);
if (!validate(req.body)) return next(badRequest(validate.errors));
```

Eight or more status codes

IMPLEMENTED

The API uses distinct outcomes including 200, 201, 400, 401, 403, 404, 409, 429, 500, 503, and 504. Status codes let clients distinguish validation, authentication, conflicts, throttling, and service failures. Gateway tests explicitly exercise common statuses; individual codes also depend on the request and service outcome.

Representations, caching, and paging

ETag and conditional 304

IMPLEMENTED

Express generates entity tags for eligible responses, and the gateway forwards If-None-Match upstream and preserves ETag on the way back. A matching conditional GET can return 304 Not Modified, allowing clients to reuse their cached copy. The gateway Supertest suite checks this path. See api-gateway/src/app.js and tests/api.test.js.

```
headers: { "if-none-match": req.get("If-None-Match") || "" }
// After the upstream response: preserve ETag; end on 304.
```

JSON and XML negotiation

PARTIAL

Product list, category, and detail GET routes inspect Accept and return JSON or XML; Vary: Accept keeps caches from mixing the representations. Other service resources are not all XML-enabled, so this is a product API capability rather than universal negotiation. See product-service/src/server.js.

```
res.vary("Accept");
if (req.accepts(["json", "xml"]) === "xml") {
  return res.type("application/xml").send(asXml(body));
}
```

Cursor pagination with a next link

IMPLEMENTED

Product and order lists encode the last row’s creation time and ID into a cursor. The next request uses after=<cursor>; the response includes pagination.nextCursor, links.next, and a Link header. This supports stable sequential browsing without large page offsets. Product cursor mode uses the default newest-first ordering; offset pagination remains available. See product-service/src/server.js and order-service/src/server.js.

```
if (next) res.set("Link", `<${next}>; rel="next"`);
return res.json({ data: items, links: { next },
  pagination: { nextCursor } });
```

Cache-Control on GET

IMPLEMENTED

The gateway sets private, no-cache by default for GET and HEAD; health endpoints use no-store. The product service sets public, max-age=30, must-revalidate for its GET/HEAD responses. Explicit policy avoids accidental long-lived caching of account data while allowing short-lived caching of catalog data. Check each route’s final headers when adding a new service.

```
if (req.method === "GET" || req.method === "HEAD")
  res.set("Cache-Control", "public, max-age=30, must-revalidate");
```

Errors, traffic, and API discovery

One consistent error body

PARTIAL

Gateway-generated errors use { success: false, message, requestId }, and upstream non-success responses are normalized to that shape at the gateway. Several services still format their own errors differently when accessed directly, and not every route is guaranteed to pass through the gateway. Treat the gateway as the consistent public boundary; service-wide unification remains follow-up work. See api-gateway/src/app.js.

```
res.status(status).json({ success: false,
  message: status >= 500 ? "Internal server error" : error.message,
  requestId: req.requestId });
```

Rate-limit headers and 429

IMPLEMENTED

express-rate-limit applies a configurable per-process request limit to API routes and emits standard rate-limit headers. Once a client exceeds the window, it receives 429. This gives clients a signal to slow down. Because counters live in each process, multiple gateway replicas do not yet share one global quota. See api-gateway/src/app.js.

```
rateLimit({ windowMs: 60_000, limit: 120,
  standardHeaders: true, legacyHeaders: false,
  handler: (req, res) => res.status(429).json(errorBody) });
```

Hand-written OpenAPI and Swagger UI

IMPLEMENTED

The API description is maintained as source code in api-gateway/src/openapi.js rather than generated from routes. The gateway serves it at /openapi.json and hosts Swagger UI at /docs. This makes the contract inspectable and provides an interactive way for developers to explore documented endpoints.

```
const swaggerDocument = require("./openapi");
app.get("/openapi.json", ...);
app.use("/docs", swaggerUi.serve, swaggerUi.setup(swaggerDocument));
```

GraphQL alongside REST

IMPLEMENTED

The gateway also exposes /graphql with a products query supporting first, after, search, and category. Its resolver calls the product service and returns a connection-like list and next cursor. GraphQL is an additional product-reading interface; the broader domain is still primarily REST. See api-gateway/src/app.js.

```
type Query { products(first: Int = 20, after: String,
  search: String, category: String): ProductConnection! }
```

Data and outbound service calls

SQL with parameterized queries + MongoDB

IMPLEMENTED

MySQL is used by the gateway for idempotency and request audit data, with mysql2 parameter placeholders and values passed separately. This keeps user-controlled values out of SQL syntax. MongoDB remains the document store for domain services such as products, orders, payments, and auth. The two databases serve different data shapes. See api-gateway/src/sql-store.js and service model files.

```
await pool.execute(
  "SELECT ... WHERE scope_hash=? AND idempotency_key=?",
  [scopeHash, idempotencyKey]
);
```

Timeout on outbound calls

IMPLEMENTED

Gateway fetch calls use AbortSignal.timeout with an environment-configured limit. Order and payment service Axios calls also set explicit timeouts. A stalled dependency therefore cannot hold a request open indefinitely; timeout failures are surfaced as gateway errors (typically 504). See api-gateway/src/app.js, order-service/src/server.js, and payment-service/src/services/payment.service.js.

```
signal: AbortSignal.timeout(
  Number(process.env.OUTBOUND_TIMEOUT_MS) || 5000
)
```

Retry with increasing backoff

IMPLEMENTED

retryWithBackoff applies increasing delays between retry attempts, with configurable retry count and base delay. The gateway limits automatic retries to GET and HEAD for REST proxy traffic, because repeating a write could create duplicate effects. GraphQL product reads can also retry. See api-gateway/src/resilience.js.

```
await retryWithBackoff(call, { retries: 2, baseDelayMs: 200 });
// Delay grows between attempts; writes are not auto-retried.
```

Circuit breaker

IMPLEMENTED

The gateway tracks transient failures per upstream service. After the configured failure threshold, calls fail quickly while the breaker is open; after a reset interval it permits a probe. This reduces repeated waits against a dependency that is already failing. It is in-memory per gateway process, not a shared distributed breaker. See api-gateway/src/resilience.js.

```
if (state.failures >= this.failureThreshold)
  state.openedAt = Date.now();
```

Security, middleware, and operations

Deliberate CORS configuration

IMPLEMENTED

Gateway and service CORS origins come from CORS_ORIGIN, parsed as a comma-separated allowlist, with credentials enabled. This lets the browser frontend make authenticated cross-origin requests while avoiding a wildcard origin. Set the variable to the actual frontend origin(s) in each environment. See api-gateway/src/app.js and service app/server files.

```
cors({ origin: (process.env.CORS_ORIGIN || defaultOrigin)
  .split(",").map(value => value.trim()), credentials: true })
```

Express middleware and central errors

IMPLEMENTED

Express middleware handles security headers, CORS, request IDs, body parsing, cache policy, throttling, validation, and routing. The gateway's final error middleware logs failures and emits the public error envelope. Services retain their own domain-level error handling, with the gateway normalizing proxied failures.

```
app.use(express.json({ limit: "200kb" }));
app.use((error, req, res, next) => {
  res.status(status).json({ success: false, message, requestId });
});
```

Structured logs and correlation ID

IMPLEMENTED

Each gateway request receives an X-Request-Id (or keeps the caller's supplied ID). That ID is returned in the response, included in JSON log records, forwarded to services, and written to the SQL audit table. It helps connect events belonging to one request across the service boundary. See api-gateway/src/app.js and sql-store.js.

```
console.log(JSON.stringify({ level: "info", service,
  requestId: req.requestId, method: req.method,
  status: res.statusCode, durationMs }));
```

Identity and runtime configuration

JWT authentication and authorization

IMPLEMENTED

Protected routes require a Bearer JWT. Services verify it using JWT_SECRET, and administrative actions check the role claim (for example, product writes require admin). The gateway checks that protected requests include an Authorization header; signature and role verification remain in the owning service. Keep JWT_SECRET private and consistent wherever tokens must be verified. See product-service/src/server.js and order-service/src/server.js.

```
req.user = jwt.verify(token, process.env.JWT_SECRET);
if (req.user.role !== "admin") return res.sendStatus(403);
```

Docker and twelve-factor configuration

IMPLEMENTED

Dockerfiles package services, and docker-compose.yml connects the gateway, domain services, MySQL, and MongoDB for local development. Ports, credentials, origins, timeouts, and service URLs are configured through environment variables (with examples in Backend/.env.example). This makes the app portable across environments. Use production secrets in the deployment platform, not committed defaults.

```
PRODUCT_SERVICE_URL: http://product-service:4002
SQL_DATABASE: ${SQL_DATABASE:-smartcart}
MONGO_URI: mongodb://mongodb:27017/smartcart
```

Configuration to confirm per environment

IMPLEMENTED

Set CORS_ORIGIN to the frontend origin, configure database URLs and credentials, JWT_SECRET, upstream service URLs, and timeout/rate-limit values before running outside local Docker. The environment sample documents names and safe local defaults; rotate secrets and provide real values in the deployment environment.

Verification and remaining work

Automated Jest and Supertest tests

IMPLEMENTED

The API gateway suite uses Jest and Supertest to exercise HTTP behavior without requiring a live external service for every case. Current coverage includes AJV rejection, versioned routes and ETags, conditional 304, idempotent replay/conflict, OpenAPI/Swagger availability, rate-limit headers, and circuit behavior. The most recent recorded run passed 6 tests. This is useful regression coverage, not proof of every deployment configuration or every downstream service integration. See api-gateway/tests/api.test.js.

```
await request(app).get("/v1/api/products")
  .set("If-None-Match", etag).expect(304);
```

Pending: live internet deployment

PENDING

A deployment that is publicly reachable is not present in this workspace. Docker Compose prepares a local multi-service stack, but a real deployment still needs a hosting target, production environment variables and secrets, database hosting, and a deployed URL. After deployment, verify the health endpoint and key routes over HTTPS.

Scope and practical caveats

IMPLEMENTED

This guide covers the checklist in the request. The implementation is strongest at the gateway boundary; some policies (rate limits and circuit state) are process-local, XML is limited to product reads, and error normalization is most reliable for requests that pass through the gateway. A production rollout should validate secret management, shared rate-limit/idempotency behavior, database readiness, and deployed service URLs.

Where to start reading

IMPLEMENTED

Backend/api-gateway/src/app.js - gateway pipeline and public behavior. Backend/api-gateway/src/resilience.js - retries and breaker. Backend/api-gateway/src/sql-store.js - SQL persistence. Backend/product-service/src/server.js - catalog negotiation and cursors. Backend/order-service/src/server.js and Backend/payment-service/src/services/payment.service.js - domain calls and payment integration. Backend/api-gateway/tests/api.test.js - gateway verification.